

MSc 2025-2026 · Data Architecture for AI

Labo Santé

Architecture de données containerisée pour un projet IA

Livrable Final · Aline Maire · 16 juin 2026

Repo : github.com/Almaire-Lab/labo-sante

Sommaire

1. Cahier des charges
2. Schémas SQL + NoSQL
3. Diagramme d'architecture
4. Extraits commentés du `docker-compose.yml`
5. Captures pgAdmin, Mongo Express, Portainer
6. Documentation d'utilisation

1. Cahier des charges

1.1 Contexte

Un **laboratoire de santé fictif** souhaite centraliser ses données médicales aujourd'hui éclatées entre plusieurs fichiers et logiciels métier. L'objectif : bâtir une plateforme **fiable, scalable, administrable, observable** et **portable**. À terme, ces données alimenteront des modèles IA (prédiction de prescription, détection d'interactions médicamenteuses, NLP sur consultations).

1.2 Données à gérer

Domaine	Type	Stockage	Justification
Médecins, Patients, Médicaments	structuré	PostgreSQL	schéma stable, intégrité forte, jointures
Prescriptions (N-N)	structuré	PostgreSQL	relations patient ↔ médecin ↔ médicament
Consultations	semi-structuré	MongoDB	symptômes/diagnostics à structure variable, pas de jointures nécessaires

1.3 Besoins fonctionnels

- **F1** — Enregistrer/consulter médecins, patients, médicaments via une UI (pgAdmin).
- **F2** — Créer des prescriptions liant patient + médecin + 1..N médicaments.
- **F3** — Stocker les consultations (symptômes libres, diagnostic structuré CIM-10) avec lien `patient_id` vers PostgreSQL.
- **F4** — Ingérer en masse des données simulées (jusqu'à 6 M de lignes) pour tester la volumétrie.
- **F5** — Re-exécuter le pipeline sans casser les données (idempotence via `ON CONFLICT`).
- **F6** — Consulter en temps réel l'état des services et leurs logs (Portainer + `docker stats`).

1.4 Besoins techniques

Catégorie	Exigence
Conteneurisation	<code>docker compose up -d</code> démarre tous les services
Persistance	volumes Docker nommés pour Postgres et Mongo (survie à <code>down</code>)
Réseau	réseau Docker isolé, communication inter-services par nom de service
Secrets	aucun mot de passe en dur — variables via fichier <code>.env</code> (jamais commité)
Ordonnancement	<code>depends_on: service_healthy</code> + healthchecks
Observabilité	pgAdmin, Mongo Express, Portainer, <code>docker stats</code>
Reproductibilité	<code>.env.example</code> , <code>requirements.txt</code> , <code>Dockerfile</code> versionnés

2. Schémas SQL + NoSQL

2.1 Schéma SQL (PostgreSQL) — export dbdiagram.io

Lien éditable : dbdiagram.io/d/Labo-Sante-Schema-SQL

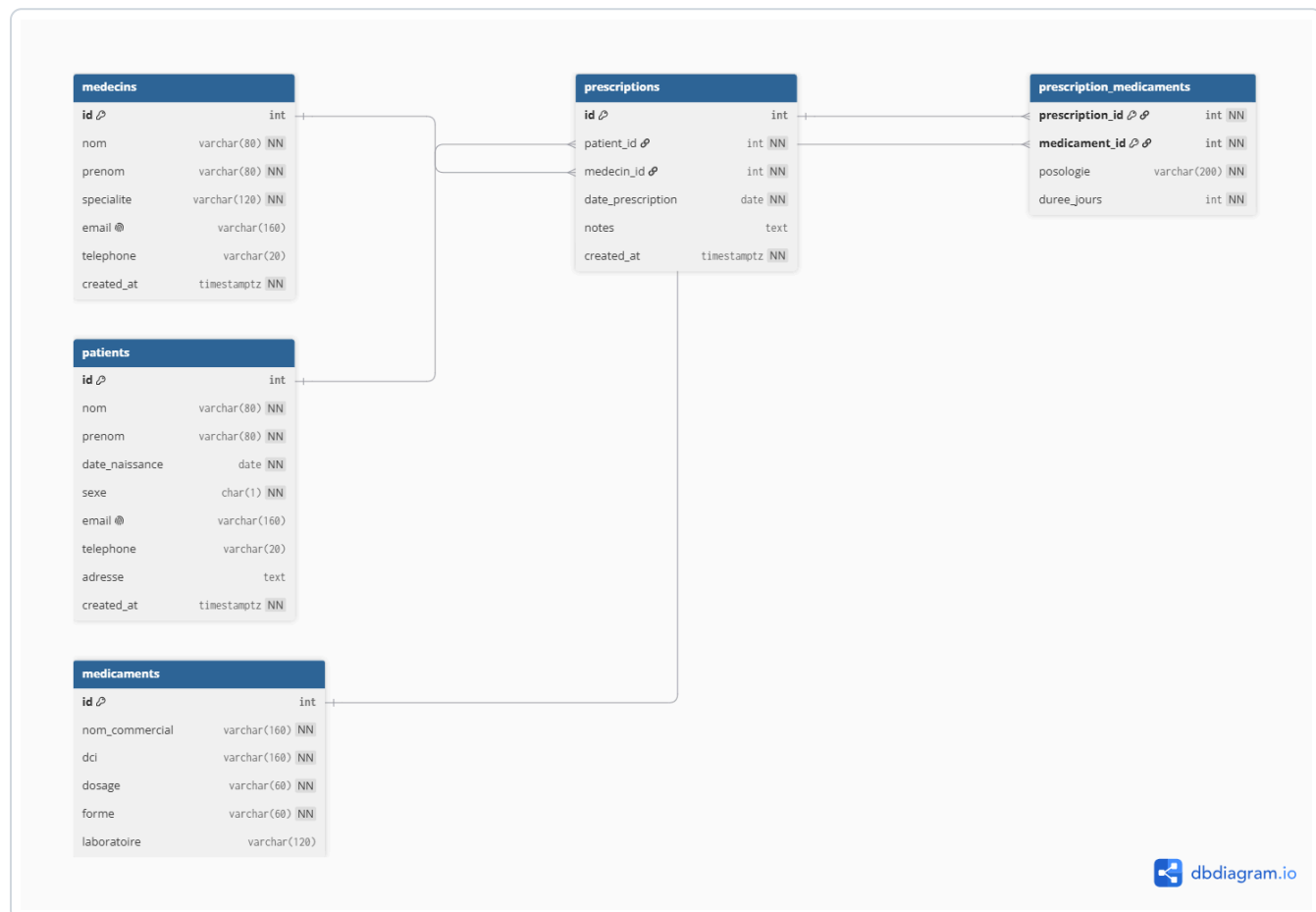


Table	Rôle	Clés / contraintes
medecins	référentiel praticiens	PK <code>id</code> , UNIQUE <code>email</code>
patients	référentiel patients	PK <code>id</code> , UNIQUE <code>email</code> , CHECK <code>sexe</code> IN ('M', 'F', 'X')
medicaments	référentiel médicaments	PK <code>id</code> , UNIQUE (<code>nom_commercial</code> , <code>dosage</code> , <code>forme</code>)
prescriptions	actes de prescription	PK <code>id</code> , FK <code>patient_id</code> , FK <code>medecin_id</code>
prescription_medicaments	liaison N-N	PK composite (<code>prescription_id</code> , <code>medicament_id</code>)

Index sur `patient_id` , `medecin_id` , `date_prescription` , `dci` . Auto-exécution au 1er démarrage via `/docker-entrypoint-initdb.d/01-schema.sql` .

2.2 Schéma NoSQL (MongoDB) — document `consultations`

```

{
  "_id": "ObjectId('...')",
  "patient_id": 1428,           // lien logique vers PostgreSQL
  "medecin_id": 12,
  "date": "2026-06-09T09:30:00Z",
  "symptoms": [                // array, longueur variable
    { "label": "toux sèche",    "duree_jours": 10, "intensite": "moderee" },
    { "label": "fièvre",       "temperature_c": 38.4 },
    { "label": "douleur thoracique", "intensite": "legere" }
  ],
  "diagnosis": {               // object, structure variable
    "primary": { "code_cim10": "J20.9", "label": "Bronchite aiguë" },
    "secondary": [{ "code_cim10": "R05", "label": "Toux" }],
    "severity": "moderee",
    "confidence": 0.82
  },
  "notes": "Auscultation : râles bronchiques bilatéraux."
}

```

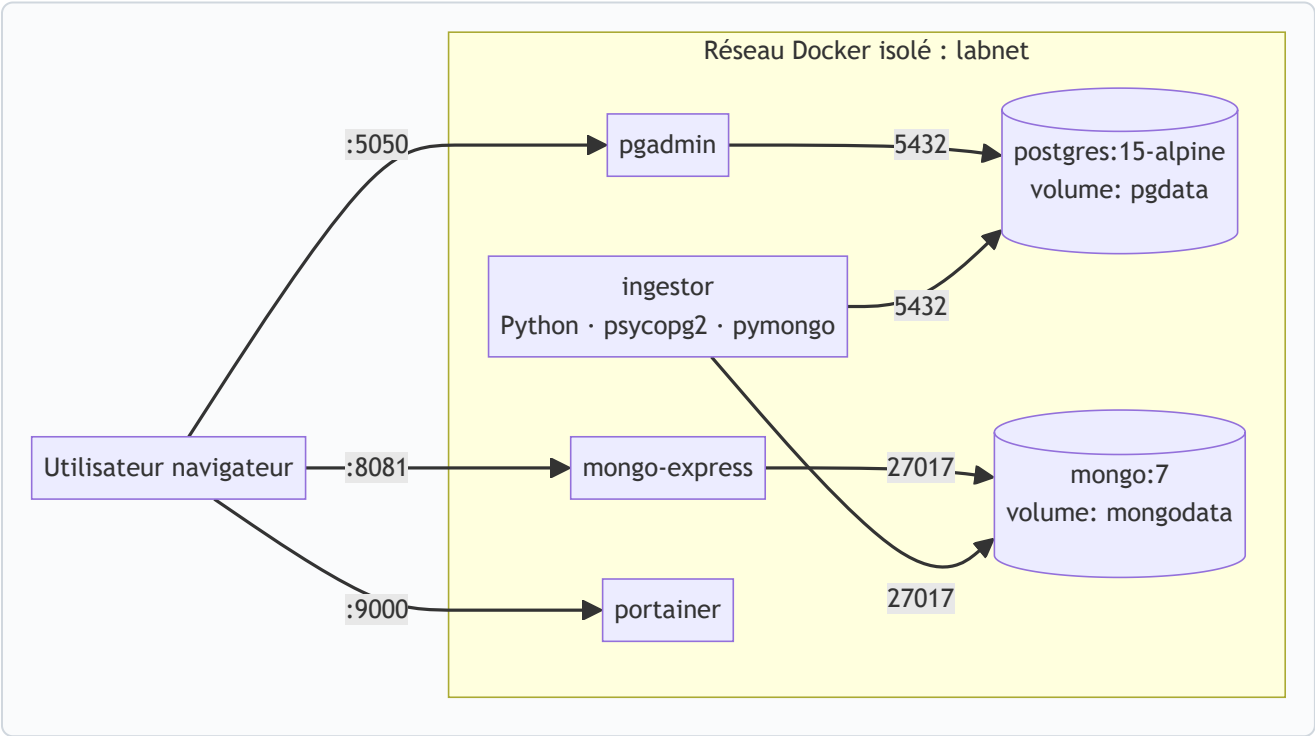
Pourquoi NoSQL ici ? `symptoms` et `diagnosis` ont une longueur et une structure variables (impossible à modéliser proprement en SQL sans 5+ tables). Pas de jointure complexe nécessaire — toute lecture se fait par `patient_id` ou `date`. `patient_id` assure le lien logique avec PostgreSQL. Évolution facile du modèle sans migration.

Index recommandés (créés à l'ingestion) :

- `{ patient_id: 1 }` — toutes les consultations d'un patient
- `{ date: -1 }` — chronologie inverse
- `{ "diagnosis.primary.code_cim10": 1 }` — recherche par diagnostic

3. Diagramme d'architecture

6 services Docker dans un **réseau isolé** `labnet`. Communication inter-services par **nom DNS** (`postgres`, `mongo` ...). Volumes nommés (`pgdata`, `mongodata`, `pgadmin`, `portainer`) pour la persistance.



Service	Image	Port hôte	Volume	Dépend de
postgres	<code>postgres:15-alpine</code>	127.0.0.1: 5432	pgdata	—
mongo	<code>mongo:7</code>	127.0.0.1: 27017	mongodata	—
pgadmin	<code>dpage/pgadmin4</code>	127.0.0.1: 5050	pgadmin	postgres (healthy)
mongo-express	<code>mongo-express:1</code>	127.0.0.1: 8081	—	mongo (healthy)
portainer	<code>portainer/portainer-ce</code>	127.0.0.1: 9000	portainer	—
ingestor	<code>build ./app</code> (Python 3.12)	—	—	postgres + mongo healthy

Healthchecks : `pg_isready` pour Postgres, `db.adminCommand({ping:1})` pour Mongo.

`docker-compose.yml`

postgres

```
image: postgres:15-alpine # image officielle, version 15, alpine = légère
environment:
  POSTGRES_USER:      ${POSTGRES_USER}      # lu depuis .env
  POSTGRES_PASSWORD:  ${POSTGRES_PASSWORD}
  POSTGRES_DB:        ${POSTGRES_DB}
ports:
  - "127.0.0.1:5432:5432" # exposé uniquement en local
volumes:
  - pgdata:/var/lib/postgresql/data # PERSISTANCE des données
  - ./sql/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
  # DDL auto-exécutée au 1er démarrage
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER -d $$POSTGRES_DB"]
  interval: 5s
  retries: 10
networks: [labnet]
```

mongo

```
image: mongo:7
environment:
  MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER}
  MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
  MONGO_INITDB_DATABASE: ${MONGO_DB}
ports: [ "127.0.0.1:27017:27017" ]
volumes:
  - mongodata:/data/db
healthcheck:
  # MongoDB 7 requiert l'auth -- ping authentifié avec exit 0/1
  test: ["CMD-SHELL", "mongosh --quiet -u ${MONGO_INITDB_ROOT_USERNAME} -p  

    ${MONGO_INITDB_ROOT_PASSWORD} --authenticationDatabase admin  

    --eval 'db.adminCommand({ping:1}).ok' | grep -q 1"]
  interval: 10s
  retries: 10
  start_period: 20s
networks: [labnet]
```

pgadmin

[illegible]

4.4 Service `ingestor` (pipeline Python)

```
ingestor:
  build: ./app                                # IMAGE CONSTRUITE LOCALEMENT
                                              # depuis app/Dockerfile
  environment:
    POSTGRES_HOST: postgres                  # nom du SERVICE, pas localhost
    MONGO_HOST:    mongo
    NB_PRESCRIPTIONS: ${NB_PRESCRIPTIONS:-60000} # paramétrable
    NB_CONSULTATIONS: ${NB_CONSULTATIONS:-60000}
  depends_on:
    postgres: { condition: service_healthy }
    mongo:    { condition: service_healthy }
  profiles: ["ingest"]                      # script one-shot, ne démarre
                                              # PAS par défaut. Lancer avec :
                                              # docker compose --profile ingest
                                              # run --rm ingestor
```

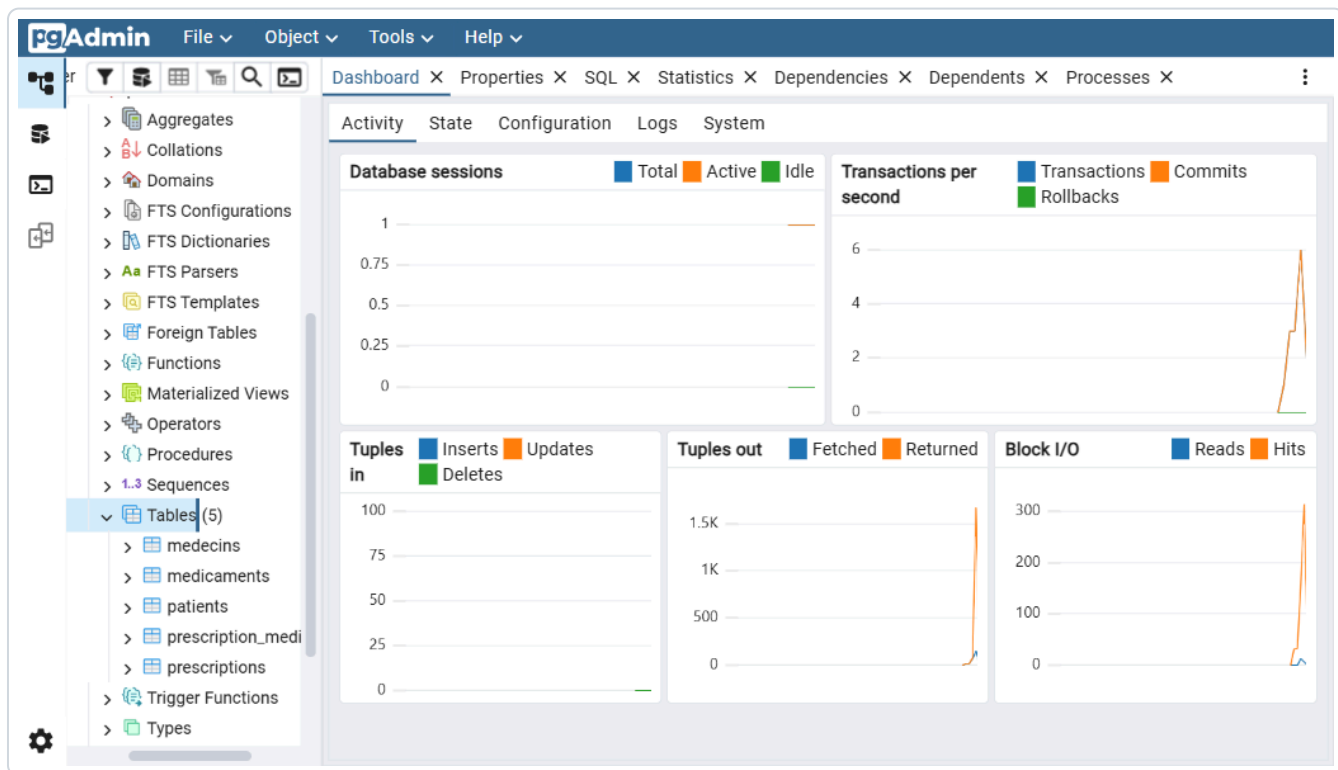
4.5 Volumes & réseaux

```
volumes:
  pgdata: {}      # persistance données Postgres
  mongodata: {}   # persistance données Mongo
  pgadmin: {}     # config pgAdmin (serveurs enregistrés)
  portainer: {}   # config Portainer (utilisateurs, dashboards)

networks:
  labnet:
    driver: bridge # réseau Docker isolé, DNS interne automatique
```

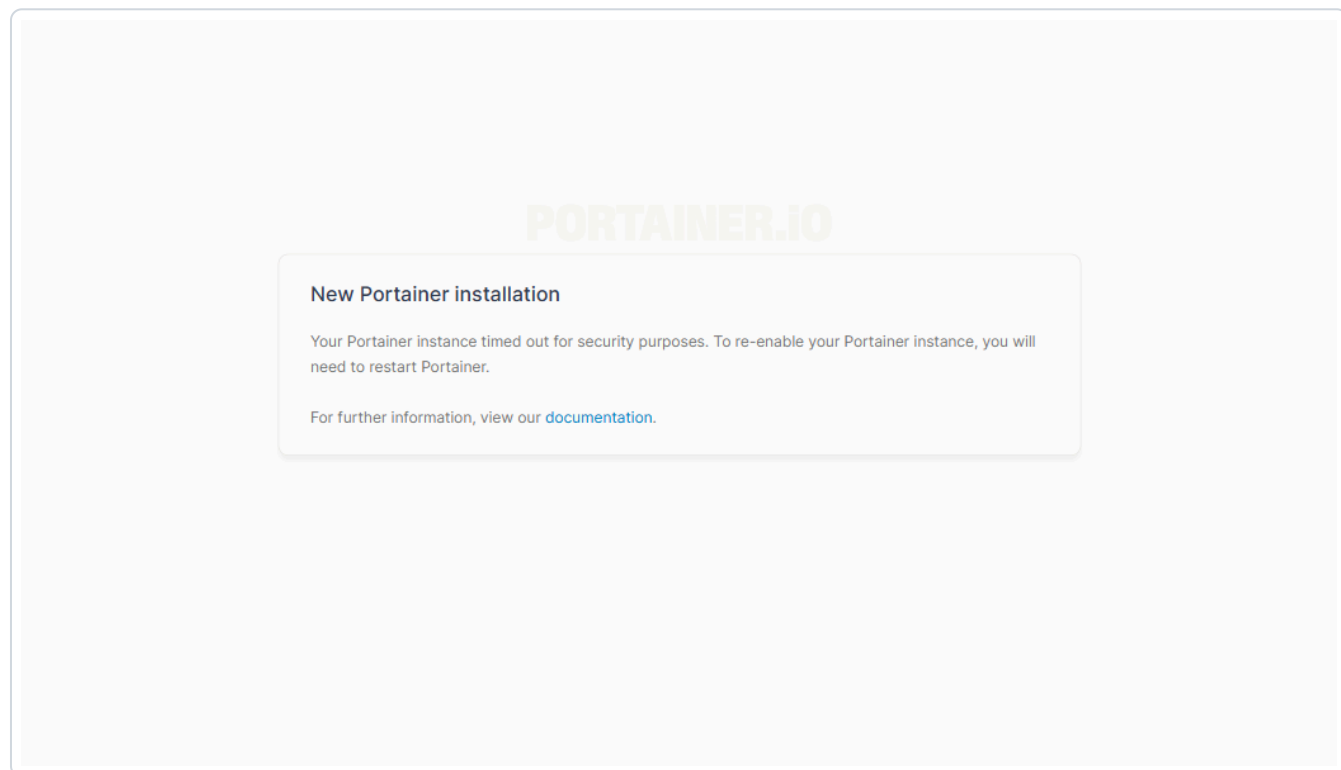

5. Captures pgAdmin, Mongo Express, Portainer

5.1 pgAdmin — serveur **labo** connecté, 5 tables visibles



Arbre Servers → labo → Databases → labo → Schemas → public → Tables (5).

5.2 Mongo Express — DB `labo` > collection `consultations`



Document type avec `patient_id` , `symptoms` , `diagnosis` et 3 index actifs.

5.3 Portainer — dashboard de la stack

The screenshot displays the Portainer.io dashboard interface. On the left is a dark sidebar with the 'PORTAINER.io' logo and 'COMMUNITY EDITION' text. Below the logo is a navigation menu with items: Home, local (selected), Dashboard, Templates, Stacks, Containers, Images, Networks, Volumes, Events, and Host. At the bottom of the sidebar, it says 'Portainer Community Edition 2.39.3 LTS'. The main area is titled 'Environment summary' and 'Dashboard'. It features a table with environment details: Environment (local - Standalone 29.5.3), URL (/var/run/docker.sock), and Tags (-). Below this are four summary cards: '1 Stack', '6 Containers' (with sub-status: 6 running, 0 stopped, 2 healthy, 0 unhealthy), '6 Images' (with 3.1 GB usage), and '5 Volumes'. A user profile 'admin' is visible in the top right corner.

Environment summary	
Environment	local - Standalone 29.5.3
URL	/var/run/docker.sock
Tags	-

Resource Type	Count	Additional Info
Stack	1	
Containers	6	6 running, 0 stopped, 2 healthy, 0 unhealthy
Images	6	3.1 GB
Volumes	5	

Vue d'ensemble : 6 conteneurs, 5 volumes, 6 images, stack **labo-sante** .

5.4 Portainer — liste des conteneurs pendant ingestion

Upgrade to Business Edition

PORTAINER.io COMMUNITY EDITION

Home

local

Dashboard

Templates

Stacks

Containers

Images

Networks

Volumes

Events

Host

Portainer Community Edition 2.39.3 LTS

Containers

Container list

Containers Search... x

Start Stop Kill Restart Pause Resume

Name	State	Quick Actions	Stack	Image
labo-mongo	healthy		labo-sante	mongo:7
labo-mongo-express	running		labo-sante	mongo-express:1
labo-pgadmin	running		labo-sante	dpage/pgadmin4:1
labo-portainer	running		labo-sante	portainer/portaine
labo-postgres	healthy		labo-sante	postgres:15-alpine
labo-sante-ingestor-run-614a9...	running		labo-sante	labo-sante/ingestor

Items per page 10

L'ingestor en **running** aux côtés des 5 services **healthy** permanents.

5.5 Portainer — CPU & Memory de **postgres** pendant ingestion

Upgrade to Business Edition

PORTAINER.io
COMMUNITY EDITION

Home

local

Dashboard

Templates

Stacks

Containers

Images

Networks

Volumes

Events

Host

Portainer Community Edition 2.39.3 LTS

Containers > labo-postgres > Stats

Container statistics

admin

About statistics

This view displays real-time statistics about the container **labo-postgres** as well as a list of the running processes inside this container.

Refresh rate 1s

Memory usage

CPU usage

Network usage (aggregate)

I/O usage (aggregate)

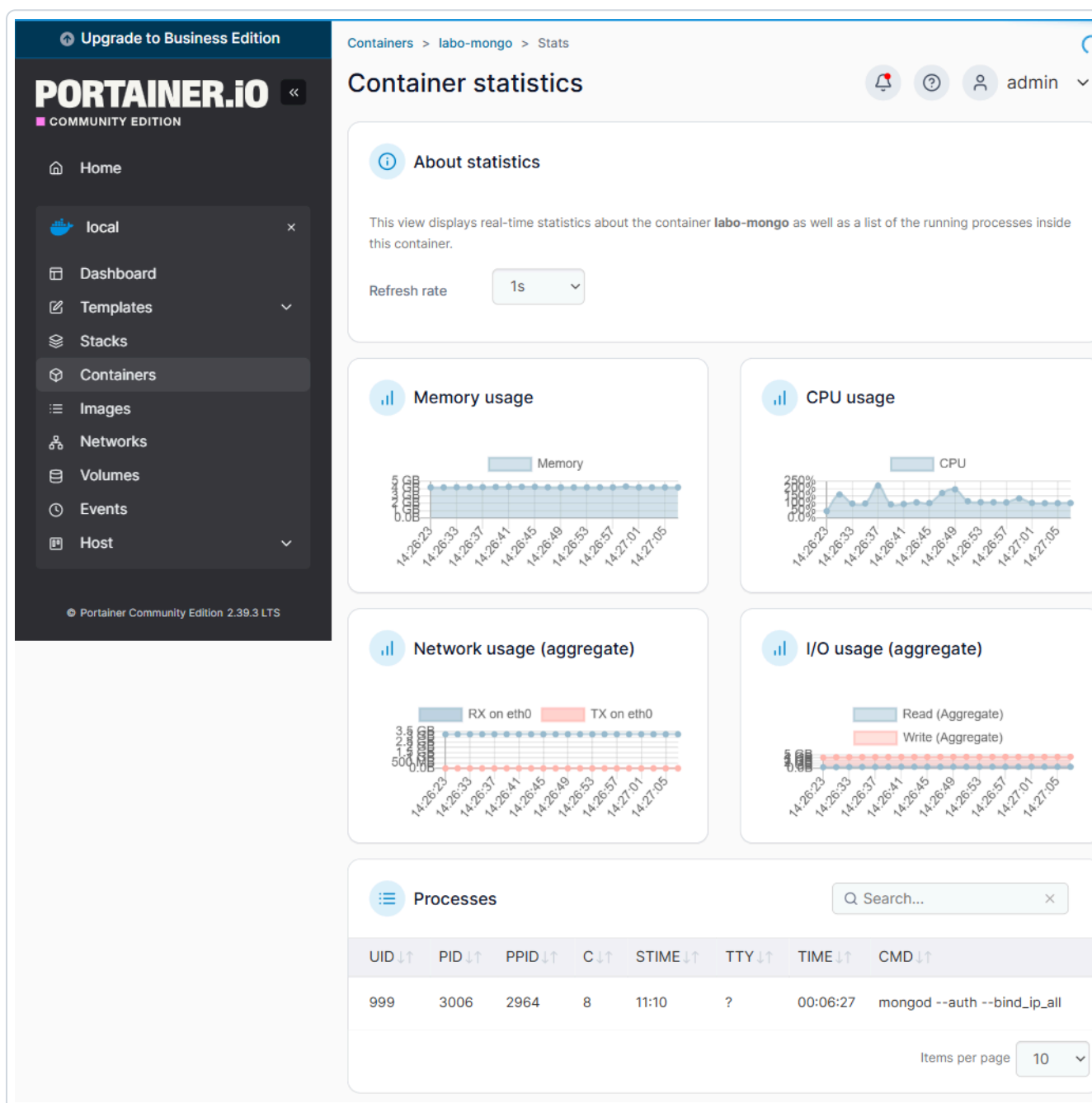
Processes

UID	PID	PPID	C	STIME	TTY	TIME	CMD
70	646	508	0	11:06	?	00:00:03	postgres
70	910	646	0	11:06	?	00:00:07	postgres: checkpointer
70	911	646	0	11:06	?	00:00:02	postgres: background writer
70	1100	646	0	11:06	?	00:00:09	postgres: walwriter
70	1101	646	0	11:06	?	00:00:00	postgres: autovacuum launcher
70	1102	646	0	11:06	?	00:00:00	postgres: logical replication launcher

Items per page 10

Refresh 1s. CPU jusqu'à 160% lors des INSERT bulk + table de liaison N-N. Liste des processus Postgres (postgres, checkpointer, walwriter, autovacuum...) visible.

5.6 Portainer — CPU & Memory de mongo pendant ingestion



Refresh 1s. CPU jusqu'à 250% pendant **insert_many** en batches de 5000 docs. Pic mémoire correspond au remplissage du cache WiredTiger.

5.7 Sortie `docker stats` pendant ingestion 6M

NAME	CPU %	MEM USAGE	BLOCK I/O
labo-sante-ingestor	25 %	56 MiB	0 B
labo-postgres	75 %	245 MiB	25 MB / 7.33 GB
labo-mongo	0.5 %	498 MiB	287 KB / 286 MB
labo-pgadmin	0.03 %	300 MiB	(idle)
labo-mongo-express	0 %	47 MiB	(idle)
labo-portainer	0 %	19 MiB	monitoring

6. Documentation d'utilisation

6.1 Démarrage rapide (de zéro)

```
# 1. Cloner le repo
git clone https://github.com/Almaire-Lab/labo-sante.git
cd labo-sante

# 2. Configurer les secrets
cp .env.example .env
# (éditer .env avec un éditeur de texte et mettre de vrais mots de passe)

# 3. Démarrer la stack (5 services)
docker compose up -d

# 4. Vérifier
docker compose ps          # tous Up, postgres + mongo healthy
docker exec labo-postgres psql -U lab_admin -d labo -c "\dt"
# -> 5 tables : medecins, patients, medicaments, prescriptions, prescription_medicaments
```

6.2 Accès aux interfaces web

Service	URL	Identifiants
pgAdmin	http://localhost:5050	PGADMIN_EMAIL / PGADMIN_PASSWORD (depuis .env)
Mongo Express	http://localhost:8081	MEX_USER / MEX_PASSWORD
Portainer	http://localhost:9000	à créer au 1 ^{er} accès

Connecter pgAdmin à Postgres : Add new server → Connection → Host = `postgres` (nom du service, pas `localhost`) — Port 5432 — User/Password = valeurs du `.env`.

6.3 Lancer une ingestion

```
# Ingestion par défaut (60 000 lignes, ~45 s)
docker compose --profile ingest run --rm ingestor

# Volumétrie personnalisée (PowerShell)
$env:NBPRESRIPTIONS = "600000"
$env:NBCONSULTATIONS = "600000"
$env:NBPATIENTS = "20000"
$env:NBMEDECINS = "200"
docker compose --profile ingest run --rm ingestor

# Pour Linux/Mac : NBPRESRIPTIONS=600000 NBCONSULTATIONS=600000 ...
# docker compose --profile ingest run --rm ingestor
```

6.4 Connexion programmatique (depuis l'hôte)

```
postgresql://lab_admin:****@localhost:5432/lab0
mongodb://lab_admin:****@localhost:27017/?authSource=admin
```


6.5 Monitoring

```
# Métriques en direct
docker stats

# Logs d'un service
docker compose logs -f postgres

# Dashboard graphique
# -> Portainer : http://localhost:9000
# Containers > [nom] > Stats : CPU, RAM, Network, I/O en temps réel
```

6.6 Arrêt / nettoyage

```
docker compose down          # stoppe, GARDE les volumes (= les données)
docker compose down -v      # ⚠ supprime AUSSI les volumes (perte totale)
```

6.7 Repo & livrables

- Dépôt GitHub : github.com/Almaire-Lab/labo-sante
- Code source : `app/` (Python) · `sql/` (DDL) · `mongo/` (schéma JSON)
- Configuration : `docker-compose.yml` · `.env.example` · `.gitignore`
- Documentation : `docs/` (cahier des charges, architecture, schémas exportés, screenshots, benchmarks)